Phase 1: Single-Qubit Foundations

Cai Yundi Elliot

Contents

1 Dirac Notation: The Language of Quantum Mechanics **3**
1.1 Historical Context and Motivation . . . 3
1.2 Physical Significance: What Does a Ket Represent? . . . 3
1.3 Ket Vectors: Representing Quantum States . . . 4
1.3.1 Definition and Physical Interpretation . . . 4
1.4 Bra Vectors: Dual Space and Inner Products . . . 4
1.4.1 Inner Product: Bra-Ket Combination . . . 4
1.5 Outer Products: Operators and Projectors . . . 5
1.6 Code: Dirac Notation Operations (Level 1) . . . 5
1.7 Exercises . . . 5

2 Qubit Representation: The Basic Unit of Quantum Information **6**
2.1 Physical Significance: What Is a Qubit? . . . 6
2.2 Mathematical Representation . . . 6
2.3 Special States . . . 7
2.3.1 Computational Basis States . . . 7
2.3.2 Superposition States . . . 7
2.3.3 Circular Polarization States . . . 7
2.4 Code: Creating Qubit States (Level 2) . . . 7
2.5 Exercises . . . 8

3 Bloch Sphere Visualization **9**
3.1 Physical Significance: Why the Bloch Sphere? . . . 9
3.2 Mathematical Parameterization . . . 9
3.3 Key Points on Bloch Sphere . . . 9
3.4 Code: Bloch Sphere Visualization (Level 3) . . . 10
3.5 Exercises . . . 10

4 Single-Qubit Gates **11**
4.1 Physical Significance: What Do Quantum Gates Do? . . . 11
4.2 X Gate (Quantum NOT) . . . 11
4.3 Y Gate . . . 11
4.4 Z Gate (Phase Flip) . . . 12
4.5 Hadamard Gate (H) . . . 12
4.6 Phase Gates (S, T) . . . 12
4.7 Exercises . . . 13

5 Quantum Measurement: Complete Mathematical Treatment 14
5.1 The Measurement Postulate . . . 14
5.2 Measurement Mathematics: Step-by-Step . . . 14
5.2.1 Before Measurement . . . 14
5.2.2 Probability of Outcome . . . 14
5.2.3 State After Measurement (Collapse) . . . 15
5.2.4 Example: Specific Calculation . . . 15
5.3 Measurement in Arbitrary Bases . . . 16
5.3.1 X-Basis Measurement . . . 16
5.3.2 Y-Basis Measurement . . . 16
5.4 Expectation Values . . . 17
5.5 Code: Custom State Creator (Level 4) . . . 18
5.6 Code: Z-Basis Measurement (Level 5) . . . 20
5.7 Measurement in Different Bases . . . 22
5.7.1 Why Rotate Before Measuring? . . . 22
5.7.2 Mathematical Framework . . . 22
5.7.3 Amplitude Projection Method . . . 23
5.8 Code: Measure Any Basis (Level 6) . . . 23
5.9 Code: Apply X Gate (Level 7) . . . 27
5.10 Code: All Single Gates (Level 8) . . . 29
5.11 Code: Gate Transformer (Level 9) . . . 32
5.12 Exercises . . . 34
5.13 Key Physical Concepts . . . 35
5.14 Why Single-Qubit Physics Matters . . . 36
5.15 Real-World Applications . . . 36

1 Dirac Notation: The Language of Quantum Mechanics

1.1 Historical Context and Motivation

Dirac notation, also known as bra-ket notation, was introduced by Paul Dirac in his seminal 1939 book "The Principles of Quantum Mechanics." This notation has since become the universal mathematical language of quantum mechanics and quantum computing.

Why Dirac notation matters:

Before Dirac's innovation, quantum mechanics was expressed using:

- **Matrix mechanics** (Heisenberg): Cumbersome matrix calculations
- **Wave mechanics** (Schrödinger): Differential equations in position space

Dirac's notation unified these approaches and revealed the underlying Hilbert space structure. Think of it as the difference between writing out longhand multiplication versus using algebraic symbols—Dirac notation lets us manipulate quantum concepts with elegance and precision.

For quantum computing specifically, Dirac notation is indispensable because:

- **Compactness**: Complex quantum states can be written in a single line
- **Clarity**: The notation makes the physics transparent
- **Universality**: All quantum computing papers and textbooks use it
- **Computational power**: It directly translates to code and calculations

1.2 Physical Significance: What Does a Ket Represent?

Key Insight: A ket $|\psi\rangle$ is not just mathematical abstraction—it represents the **complete physical state** of a quantum system.

Physical Meaning:

- **Information content**: $|\psi\rangle$ encodes everything knowable about the system
- **Measurement predictions**: From $|\psi\rangle$, we can compute probabilities of all possible measurement outcomes
- **Time evolution**: $|\psi\rangle$ evolves deterministically via the Schrödinger equation
- **Superposition**: $|\psi\rangle$ can be a linear combination of basis states (unlike classical states)

Real-world examples:

- **Electron spin**: $|\psi\rangle = \alpha|\uparrow\rangle + \beta|\downarrow\rangle$ describes the electron's spin orientation
- **Photon polarization**: $|\psi\rangle = \frac{|H\rangle+|V\rangle}{\sqrt{2}}$ describes 45° polarized light
- **Qubit**: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ stores quantum information in a quantum computer
- **Atomic energy levels**: $|\psi\rangle = c_1|E_1\rangle + c_2|E_2\rangle$ describes an atom in superposition of energy states

1.3 Ket Vectors: Representing Quantum States

1.3.1 Definition and Physical Interpretation

A **ket** is written as $|\psi\rangle$ (pronounced "ket psi") and represents a vector in a complex Hilbert space. In physics terms, a ket describes the *complete state* of a quantum system—everything that can be known about the system is encoded in this vector.

For a qubit, we define two basis states:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

A general superposition state:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where $\alpha, \beta \in \mathbb{C}$ are complex probability amplitudes.

1.4 Bra Vectors: Dual Space and Inner Products

A **bra** is written as $\langle\psi|$ and represents a linear functional (a row vector) in the dual Hilbert space.

Mathematical relation:

$$\langle\psi| = (|\psi\rangle)^\dagger$$

where $\dagger$ denotes the Hermitian conjugate (conjugate transpose).

For example, if $|\psi\rangle = \begin{pmatrix} a \\ b \end{pmatrix}$, then $\langle\psi| = \begin{pmatrix} a^* & b^* \end{pmatrix}$.

1.4.1 Inner Product: Bra-Ket Combination

The inner product of two states:

$$\langle\phi|\psi\rangle = \text{complex number}$$

Example calculation:

If $|\psi\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $|\phi\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, then:

$$\langle\phi|\psi\rangle = \begin{pmatrix} 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = 0$$

This shows $|0\rangle$ and $|1\rangle$ are orthogonal.

Properties:

- **Conjugate symmetry**: $\langle\phi|\psi\rangle = \langle\psi|\phi\rangle^*$
- **Linearity**: $\langle\phi|(\alpha|\psi_1\rangle + \beta|\psi_2\rangle) = \alpha\langle\phi|\psi_1\rangle + \beta\langle\phi|\psi_2\rangle$
- **Positive definiteness**: $\langle\psi|\psi\rangle \geq 0$, equals 0 only if $|\psi\rangle = 0$
- **Normalization**: $\langle\psi|\psi\rangle = 1$ for physical states

1.5 Outer Products: Operators and Projectors

The outer product $|\psi\rangle\langle\phi|$ is an **operator** (matrix).

Example: Computational basis completeness:

$$|0\rangle\langle 0| + |1\rangle\langle 1| = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

1.6 Code: Dirac Notation Operations (Level 1)

```python
import numpy as np

# Create ket vectors
ket_0 = np.array([1, 0])  # |0>
ket_1 = np.array([0, 1])  # |1>

# Create bra vectors (conjugate transpose)
bra_0 = ket_0.conj().T  # <0|
bra_1 = ket_1.conj().T  # <1|

# Inner product (bra-ket)
inner_00 = bra_0 @ ket_0  # <0|0> = 1
inner_01 = bra_0 @ ket_1  # <0|1> = 0

print("<0|0> =", inner_00)  # Should be 1
print("<0|1> =", inner_01)  # Should be 0 (orthogonal)

# Outer product (ket-bra)
outer_00 = np.outer(ket_0, bra_0)  # |0><0|
outer_11 = np.outer(ket_1, bra_1)  # |1><1|

# Completeness relation
completeness = outer_00 + outer_11
print("\n|0><0| + |1><1| =")
print(completeness)  # Should be identity matrix
```

Listing 1: Dirac notation operations in Python

1.7 Exercises

1. Write the bra vector corresponding to $|\psi\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ i \end{pmatrix}$.
2. Calculate $\langle\psi|\psi\rangle$ and verify normalization.
3. Compute the outer product $|\psi\rangle\langle\psi|$.
4. Show that $|+\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$ is normalized.

2 Qubit Representation: The Basic Unit of Quantum Information

2.1 Physical Significance: What Is a Qubit?

Key Insight: A qubit is the quantum analogue of a classical bit, but with profoundly different properties.

Physical Meaning:

- **Two-level system**: Any quantum system with two distinguishable states
- **Superposition**: Unlike classical bits (0 OR 1), qubits can be 0 AND 1 simultaneously
- **Continuous parameters**: A qubit state requires two continuous complex numbers
- **Measurement collapse**: Upon measurement, superposition collapses to definite outcome
- **Interference**: Probability amplitudes can interfere constructively or destructively
- **Phase**: Relative phase affects interference patterns

Classical bit vs Quantum bit:

Classical Bit	**Quantum Bit (Qubit)**
State: 0 OR 1	State: $\alpha\|0\rangle + \beta\|1\rangle$
Discrete (2 states)	Continuous (infinite states)
Deterministic evolution	Unitary evolution (reversible)
Measurement reveals state	Measurement collapses state
No interference	Interference effects
Can be copied	Cannot be cloned

Real-world implementations:

- **Superconducting qubits**: IBM, Google use LC circuits
- **Trapped ions**: Electromagnetic traps hold ions
- **Photonic qubits**: Polarization encoding
- **Spin qubits**: Electron spins in silicon/diamond
- **Atomic qubits**: Neutral atoms in optical tweezers

2.2 Mathematical Representation

A general qubit state is:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$.

Alternative parameterization (Bloch sphere):

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle$$

2.3 Special States

2.3.1 Computational Basis States

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

2.3.2 Superposition States

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

2.3.3 Circular Polarization States

$$|+i\rangle = \frac{|0\rangle + i|1\rangle}{\sqrt{2}}, \quad |-i\rangle = \frac{|0\rangle - i|1\rangle}{\sqrt{2}}$$

2.4 Code: Creating Qubit States (Level 2)

```python
import numpy as np

# Computational basis states
zero = np.array([1, 0])  # |0>
one = np.array([0, 1])   # |1>

# Superposition states
plus = np.array([1/np.sqrt(2), 1/np.sqrt(2)])  # |+>
minus = np.array([1/np.sqrt(2), -1/np.sqrt(2)])  # |->

# Circular states
plus_i = np.array([1/np.sqrt(2), 1j/np.sqrt(2)])  # |+i>
minus_i = np.array([1/np.sqrt(2), -1j/np.sqrt(2)])  # |-i>

# Verify normalization
print("|0> norm:", np.linalg.norm(zero))
print("|+> norm:", np.linalg.norm(plus))

# Verify orthogonality
print("<0|1>:", np.vdot(zero, one))
print("<+|->:", np.vdot(plus, minus))

# Create custom state
alpha = 0.6
beta = 0.8j
custom = np.array([alpha, beta])
print("Custom state norm:", np.linalg.norm(custom))
```

Listing 2: Create and manipulate qubit states

2.5 Exercises

1. Verify that $|+\rangle$ is normalized.
2. Show that $\langle +|-\rangle = 0$.
3. Express $|0\rangle$ in the $\{|+\rangle, |-\rangle\}$ basis.
4. Calculate the probability of measuring $|+\rangle$ in state $\frac{3|0\rangle+4i|1\rangle}{5}$.

3 Bloch Sphere Visualization

3.1 Physical Significance: Why the Bloch Sphere?

Key Insight: The Bloch sphere provides geometric intuition for single-qubit states.

Physical Meaning:

- **Visual representation**: Abstract complex amplitudes become concrete 3D directions
- **State evolution**: Unitary operations become rotations on the sphere
- **Measurement**: Measurement axes correspond to spatial directions (X, Y, Z)
- **Orthogonality**: Orthogonal states are antipodal (opposite) on the sphere
- **Phase**: Azimuthal angle ϕ represents relative phase
- **Population**: Polar angle θ determines $|0\rangle$ vs $|1\rangle$ probability

3.2 Mathematical Parameterization

Any pure qubit state can be written as:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle$$

The corresponding Bloch vector is:

$$\vec{r} = (\sin\theta\cos\phi, \sin\theta\sin\phi, \cos\theta)$$

Key points:

- $\theta \in [0, \pi]$: Polar angle (0 = north pole, π = south pole)
- $\phi \in [0, 2\pi)$: Azimuthal angle (longitude)
- Factor of $1/2$: Full rotation in Hilbert space = half rotation on Bloch sphere
- Global phase irrelevant: Only relative phase ϕ matters

3.3 Key Points on Bloch Sphere

North Pole: $|0\rangle$ ($\theta = 0$, $\vec{r} = (0, 0, 1)$)
South Pole: $|1\rangle$ ($\theta = \pi$, $\vec{r} = (0, 0, -1)$)
X-axis: $|+\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$ ($\theta = \pi/2, \phi = 0$)
Negative X-axis: $|-\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}}$ ($\theta = \pi/2, \phi = \pi$)
Y-axis: $|+i\rangle = \frac{|0\rangle+i|1\rangle}{\sqrt{2}}$ ($\theta = \pi/2, \phi = \pi/2$)
Negative Y-axis: $|-i\rangle = \frac{|0\rangle-i|1\rangle}{\sqrt{2}}$ ($\theta = \pi/2, \phi = 3\pi/2$)

3.4 Code: Bloch Sphere Visualization (Level 3)

```python
import numpy as np
from qiskit.visualization import plot_bloch_multivector
from qiskit import QuantumCircuit

# Create states
zero = np.array([1, 0])
one = np.array([0, 1])
plus = np.array([1/np.sqrt(2), 1/np.sqrt(2)])
plus_i = np.array([1/np.sqrt(2), 1j/np.sqrt(2)])

# Using Qiskit's Bloch sphere visualization
qc = QuantumCircuit(1)
qc.h(0)  # Prepare |+> state
plot_bloch_multivector(qc)

# Manual Bloch vector calculation
def bloch_vector(psi):
    """Calculate Bloch vector from state vector."""
    X = np.array([[0, 1], [1, 0]])
    Y = np.array([[0, -1j], [1j, 0]])
    Z = np.array([[1, 0], [0, -1]])

    rho = np.outer(psi, psi.conj())
    rx = np.trace(rho @ X).real
    ry = np.trace(rho @ Y).real
    rz = np.trace(rho @ Z).real

    return np.array([rx, ry, rz])

print("Bloch vector of |0>:", bloch_vector(zero))
print("Bloch vector of |+>:", bloch_vector(plus))
print("Bloch vector of |+i>:", bloch_vector(plus_i))
```

Listing 3: Visualize qubit states on Bloch sphere

3.5 Exercises

1. What are the Bloch coordinates of $|1\rangle$?
2. Where on the Bloch sphere is the state $\frac{\sqrt{3}}{2}|0\rangle + \frac{1}{2}|1\rangle$?
3. Show that antipodal points correspond to orthogonal states.
4. Draw the trajectory of $|0\rangle$ under X gate rotation.

4 Single-Qubit Gates

4.1 Physical Significance: What Do Quantum Gates Do?

Key Insight: Quantum gates are unitary transformations that rotate the Bloch sphere.

Physical Meaning:

- **Reversibility**: All quantum gates are reversible
- **Information conservation**: Unitary operations preserve total probability
- **Continuous control**: Gates can rotate by any angle
- **Interference manipulation**: Gates change relative phases
- **Deterministic evolution**: Gates evolve states predictably

4.2 X Gate (Quantum NOT)

Matrix:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Action:

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle$$

Physical Significance:

- **Bit flip**: Swaps $|0\rangle \leftrightarrow |1\rangle$
- **180° rotation**: Around X-axis on Bloch sphere
- **Involutory**: $X^2 = I$
- **Hermitian**: $X = X^\dagger$
- **Pauli matrix**: One of three Pauli operators

4.3 Y Gate

Matrix:

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

Action:

$$Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle$$

Physical Significance:

- **Bit + phase flip**: Combines X and Z operations
- **180° rotation**: Around Y-axis on Bloch sphere
- **Involutory**: $Y^2 = I$
- **Hermitian**: $Y = Y^\dagger$

4.4 Z Gate (Phase Flip)

Matrix:

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Action:

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle$$

Physical Significance:

- **Phase flip**: Changes relative phase without affecting probabilities
- **180° rotation**: Around Z-axis on Bloch sphere
- **Involutory**: $Z^2 = I$
- **Hermitian**: $Z = Z^\dagger$
- **Diagonal**: Doesn't mix basis states

4.5 Hadamard Gate (H)

Matrix:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Action:

$$H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle$$
$$H|+\rangle = |0\rangle, \quad H|-\rangle = |1\rangle$$

Physical Significance:

- **Superposition creator**: Transforms basis states to equal superpositions
- **Basis change**: Rotates Z-basis to X-basis
- **Interference enabler**: Essential for quantum interference effects
- **Algorithm primitive**: Used in nearly all quantum algorithms
- **Self-inverse**: $H^2 = I$
- **Hermitian**: $H = H^\dagger$

4.6 Phase Gates (S, T)

S Gate (Phase gate, $\sqrt{Z}$):

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

T Gate ($\sqrt{S}$, $\pi/8$ gate):

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$$

Physical Significance:

- **Fractional rotations**: S = 90° around Z, T = 45° around Z
- **Phase accumulation**: Models natural phase evolution over time
- **Universal set**: T gate + Clifford gates = universal quantum computation
- **Non-Clifford**: T gate is outside Clifford group

4.7 Exercises

1. Show that $H|0\rangle = |+\rangle$.
2. Verify that $X^2 = I$.
3. Calculate HZH and identify the result.
4. Show that $S^2 = Z$.
5. Prove that H is unitary: $H^\dagger H = I$.

5 Quantum Measurement: Complete Mathematical Treatment

5.1 The Measurement Postulate

Postulate 3 (Nielsen & Chuang): A quantum measurement is described by a collection of **measurement operators** $\{M_m\}$, where:

- m labels the possible measurement outcomes
- Each M_m is a linear operator acting on the state space
- The operators satisfy the **completeness relation**:

$$\sum_m M_m^\dagger M_m = I$$

5.2 Measurement Mathematics: Step-by-Step

5.2.1 Before Measurement

Initial state:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where $|\alpha|^2 + |\beta|^2 = 1$.

Physical meaning: The system is in a superposition of both basis states simultaneously.

5.2.2 Probability of Outcome

The probability of obtaining outcome m is:

$$p(m) = \langle\psi|M_m^\dagger M_m|\psi\rangle$$

For projective measurement in the Z-basis:

- $M_0 = |0\rangle\langle 0|$ (projector onto $|0\rangle$)
- $M_1 = |1\rangle\langle 1|$ (projector onto $|1\rangle$)

Calculation for outcome 0:

$$\begin{aligned}
p(0) &= \langle\psi|M_0^\dagger M_0|\psi\rangle && (1)\\
&= \langle\psi|(|0\rangle\langle 0|)^\dagger(|0\rangle\langle 0|)|\psi\rangle && (2)\\
&= \langle\psi|0\rangle\langle 0|0\rangle\langle 0|\psi\rangle && (3)\\
&= \langle\psi|0\rangle \cdot 1 \cdot \langle 0|\psi\rangle && (4)\\
&= |\langle 0|\psi\rangle|^2 && (5)\\
&= |\alpha|^2 && (6)
\end{aligned}$$

Physical meaning: The probability equals the squared magnitude of the amplitude.

5.2.3 State After Measurement (Collapse)

Key Formula: If outcome m is obtained, the post-measurement state is:

$$|\psi'\rangle = \frac{M_m|\psi\rangle}{\sqrt{p(m)}}$$

Why normalization? The $\sqrt{p(m)}$ ensures $\langle\psi'|\psi'\rangle = 1$.
Derivation for outcome 0:

$$\begin{aligned}
|\psi'\rangle &= \frac{M_0|\psi\rangle}{\sqrt{p(0)}} && (7)\\
&= \frac{|0\rangle\langle 0|(\alpha|0\rangle + \beta|1\rangle)}{\sqrt{|\alpha|^2}} && (8)\\
&= \frac{\alpha|0\rangle\langle 0|0\rangle + \beta|0\rangle\langle 0|1\rangle}{|\alpha|} && (9)\\
&= \frac{\alpha|0\rangle \cdot 1 + \beta|0\rangle \cdot 0}{|\alpha|} && (10)\\
&= \frac{\alpha|0\rangle}{|\alpha|} && (11)
\end{aligned}$$

Physical interpretation:

- The $\beta|1\rangle$ component is **destroyed** (gone forever)
- Only the $\alpha|0\rangle$ component survives
- The phase factor $\alpha/|\alpha|$ is a global phase (physically irrelevant)
- Result: $|\psi'\rangle = |0\rangle$ (up to global phase)

Key insight: Measurement is **irreversible**—you cannot recover β from $|0\rangle$.

5.2.4 Example: Specific Calculation

Initial state: $|\psi\rangle = 0.6|0\rangle + 0.8i|1\rangle$
Probabilities:

$$\begin{aligned}
p(0) &= |0.6|^2 = 0.36 \quad (36\%) && (12)\\
p(1) &= |0.8i|^2 = 0.64 \quad (64\%) && (13)
\end{aligned}$$

If outcome 0 is measured:

$$\begin{aligned}
|\psi'\rangle &= \frac{M_0|\psi\rangle}{\sqrt{p(0)}} && (14)\\
&= \frac{|0\rangle\langle 0|(0.6|0\rangle + 0.8i|1\rangle)}{0.6} && (15)\\
&= \frac{0.6|0\rangle}{0.6} = |0\rangle && (16)
\end{aligned}$$

If outcome 1 is measured:

$$|\psi'\rangle = \frac{M_1|\psi\rangle}{\sqrt{p(1)}} \tag{17}$$

$$= \frac{|1\rangle\langle 1|(0.6|0\rangle + 0.8i|1\rangle)}{0.8} \tag{18}$$

$$= \frac{0.8i|1\rangle}{0.8} = i|1\rangle \tag{19}$$

Note: $i|1\rangle$ is physically equivalent to $|1\rangle$ (global phase).

5.3 Measurement in Arbitrary Bases

5.3.1 X-Basis Measurement

X-basis eigenstates:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Measurement operators:

$$M_+ = |+\rangle\langle +|, \quad M_- = |-\rangle\langle -|$$

Example: Measure $|0\rangle$ in X-basis

$$p(+) = |\langle +|0\rangle|^2 = \left|\frac{\langle 0|0\rangle + \langle 1|0\rangle}{\sqrt{2}}\right|^2 = \left|\frac{1+0}{\sqrt{2}}\right|^2 = \frac{1}{2} \tag{20}$$

$$p(-) = |\langle -|0\rangle|^2 = \left|\frac{\langle 0|0\rangle - \langle 1|0\rangle}{\sqrt{2}}\right|^2 = \left|\frac{1-0}{\sqrt{2}}\right|^2 = \frac{1}{2} \tag{21}$$

Post-measurement:

- If $+$: state collapses to $|+\rangle$
- If $-$: state collapses to $|-\rangle$

Physical meaning: Even though $|0\rangle$ is definite in Z-basis, it's maximally uncertain in X-basis.

5.3.2 Y-Basis Measurement

Y-basis eigenstates:

$$|+i\rangle = \frac{|0\rangle + i|1\rangle}{\sqrt{2}}, \quad |-i\rangle = \frac{|0\rangle - i|1\rangle}{\sqrt{2}}$$

These are eigenstates of the Y Pauli matrix with eigenvalues $+1$ and -1.

5.4 Expectation Values

The **expectation value** of an observable A:

$$\langle A \rangle = \langle \psi | A | \psi \rangle$$

Physical meaning: Average measurement result over many identically prepared systems.

Example: $\langle Z \rangle$ for $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$

$$\begin{aligned}
\langle Z \rangle &= \langle \psi | Z | \psi \rangle && (22)\\
&= (\alpha^* \langle 0| + \beta^* \langle 1|) Z (\alpha|0\rangle + \beta|1\rangle) && (23)\\
&= (\alpha^* \langle 0| + \beta^* \langle 1|)(\alpha|0\rangle - \beta|1\rangle) && (24)\\
&= |\alpha|^2 \langle 0|0\rangle - |\beta|^2 \langle 1|1\rangle && (25)\\
&= |\alpha|^2 - |\beta|^2 && (26)
\end{aligned}$$

Interpretation:

- If $|\psi\rangle = |0\rangle$: $\langle Z \rangle = 1$ (always measure +1)
- If $|\psi\rangle = |1\rangle$: $\langle Z \rangle = -1$ (always measure -1)
- If $|\psi\rangle = |+\rangle$: $\langle Z \rangle = 0$ (equal probability of +1 and -1)

5.5 Code: Custom State Creator (Level 4)

File: `code/foundations/04_custom_state.py`

```python
#!/usr/bin/env python3
"""
Level 4: Custom State Creator - Generalized

Now we generalize! Instead of hardcoding specific states,
we create a function that can make ANY single-qubit state.

Learning goal: From task-specific to general tool!
"""

import numpy as np
from qiskit.quantum_info import Statevector


def create_state(alpha, beta):
    """
    Create any single-qubit state from amplitudes.

    Args:
        alpha: Complex amplitude for |0>
        beta: Complex amplitude for |1>

    Returns:
        Statevector object (normalized)
    """
    # Normalize (ensure |a|^2 + |b|^2 = 1)
    norm = np.sqrt(np.abs(alpha)**2 + np.abs(beta)**2)
    alpha_norm = alpha / norm
    beta_norm = beta / norm

    # Create and return state
    psi = Statevector([alpha_norm, beta_norm])
    return psi


# Example 1: Create |0>
psi0 = create_state(1, 0)
print("1. |0> state:")
print(psi0)

# Example 2: Create |1>
psi1 = create_state(0, 1)
print("\n2. |1> state:")
print(psi1)

# Example 3: Create |+>
psi_plus = create_state(1/np.sqrt(2), 1/np.sqrt(2))
print("\n3. |+> state:")
```

```python
print(psi_plus)

# Example 4: Create custom state
psi_custom = create_state(0.6, 0.8)
print("\n4. Custom state (a=0.6, b=0.8):")
print(psi_custom)
print("   Probability of 0:", abs(psi_custom[0])**2)
print("   Probability of 1:", abs(psi_custom[1])**2)
```

Listing 4: Create any single-qubit state (Level 4)

What This Code Does:

This code generalizes state creation by defining a `create_state()` function that accepts any complex amplitudes α and β. It automatically normalizes the state to ensure $|\alpha|^2 + |\beta|^2 = 1$, then returns a Qiskit Statevector. The examples demonstrate creating basis states ($|0\rangle$, $|1\rangle$), superposition ($|+\rangle$), and custom states with specified probabilities. This transitions from hardcoded states (Levels 1-3) to a reusable tool that can create any single-qubit state.

Physical Significance:

- **Generalization**: From hardcoded states to universal state creator
- **Normalization**: Physical states must have total probability = 1
- **Complex amplitudes**: Supports both real and imaginary components
- **Reusability**: One function creates infinite possible states
- **Progression**: Level 4 transitions from copy-paste to functions

Expected Output:

```
1. |0> state:
Statevector([1.+0.j, 0.+0.j])

2. |1> state:
Statevector([0.+0.j, 1.+0.j])

3. |+> state:
Statevector([0.707+0.j, 0.707+0.j])

4. Custom state (a=0.6, b=0.8):
Statevector([0.6+0.j, 0.8+0.j])
   Probability of 0: 0.36
   Probability of 1: 0.64
```

5.6 Code: Z-Basis Measurement (Level 5)

File: `code/foundations/05_measure_z.py`

```python
#!/usr/bin/env python3
"""
Level 5: Measure in Z Basis - Task Specific

Measures a state in the Z basis (computational basis).
Task-specific: only Z basis measurement.

Learning goal: Understand Born rule and measurement probabilities
    !
"""

import numpy as np
from qiskit.quantum_info import Statevector


def measure_z(psi):
    """
    Measure state in Z basis.

    Args:
        psi: Statevector

    Returns:
        Dictionary with probabilities for |0> and |1>
    """
    prob_0 = np.abs(psi[0])**2  # |<0|psi>|^2
    prob_1 = np.abs(psi[1])**2  # |<1|psi>|^2

    return {'0': prob_0, '1': prob_1}


# Test with |+> state
plus = Statevector.from_label('+')
print("Measuring |+> in Z basis:")
results = measure_z(plus)
print(f"   P(0) = {results['0']:.2f} (50%)")
print(f"   P(1) = {results['1']:.2f} (50%)")

# Test with |0> state
zero = Statevector.from_label('0')
print("\nMeasuring |0> in Z basis:")
results = measure_z(zero)
print(f"   P(0) = {results['0']:.2f} (100%)")
print(f"   P(1) = {results['1']:.2f} (0%)")
```

Listing 5: Measure in Z basis (Level 5)

What This Code Does:

This code implements the Born rule for Z-basis measurement. The `measure_z()`

function computes probabilities $P(0) = |\psi[0]|^2$ and $P(1) = |\psi[1]|^2$ directly from the statevector amplitudes. Testing with $|+\rangle$ shows equal 50/50 probabilities (superposition), while $|0\rangle$ gives deterministic 100/0 outcomes (eigenstate). This demonstrates how measurement probabilities emerge from the quantum state's complex amplitudes.

5.7 Measurement in Different Bases

Key Principle: To measure in a basis, you must **rotate to that basis first**, then measure in the computational (Z) basis.

5.7.1 Why Rotate Before Measuring?

Quantum computers natively measure in the **Z basis** (computational basis $\{|0\rangle, |1\rangle\}$). To measure in other bases, we apply unitary gates that rotate the basis states to align with Z:

- **X basis** $\{|+\rangle, |-\rangle\}$: Apply **H gate** before measuring
- **Y basis** $\{|+i\rangle, |-i\rangle\}$: Apply **S† then H** before measuring
- **Z basis** $\{|0\rangle, |1\rangle\}$: Measure directly (no rotation needed)

5.7.2 Mathematical Framework

Born Rule in Arbitrary Basis:
For a state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ and basis $\{|b_1\rangle, |b_2\rangle\}$:

$$P(b_i) = |\langle b_i|\psi\rangle|^2$$

X Basis Measurement:
X basis states:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

To measure $|\psi\rangle$ in X basis:

1. Apply H gate: $H|\psi\rangle$
2. Measure in Z basis
3. Outcome 0 $\rightarrow$ state was $|+\rangle$
4. Outcome 1 $\rightarrow$ state was $|-\rangle$

Proof: H gate maps X basis to Z basis:

$$H|+\rangle = |0\rangle, \quad H|-\rangle = |1\rangle$$

So measuring $H|\psi\rangle$ in Z basis is equivalent to measuring $|\psi\rangle$ in X basis.
Y Basis Measurement:
Y basis states:

$$|+i\rangle = \frac{|0\rangle + i|1\rangle}{\sqrt{2}}, \quad |-i\rangle = \frac{|0\rangle - i|1\rangle}{\sqrt{2}}$$

To measure $|\psi\rangle$ in Y basis:

1. Apply S† (phase conjugate): $S^\dagger|\psi\rangle$
2. Apply H gate: $HS^\dagger|\psi\rangle$
3. Measure in Z basis

4. Outcome 0 → state was $|+i\rangle$
5. Outcome 1 → state was $|-i\rangle$

Proof: $HS^\dagger$ maps Y basis to Z basis:

$$HS^\dagger|+i\rangle = |0\rangle, \quad HS^\dagger|-i\rangle = |1\rangle$$

5.7.3 Amplitude Projection Method

Alternative approach: directly compute amplitudes in the target basis:

X Basis:

$$\langle +|\psi\rangle = \frac{\alpha+\beta}{\sqrt{2}}, \quad \langle -|\psi\rangle = \frac{\alpha-\beta}{\sqrt{2}}$$

$$P(+) = \left|\frac{\alpha+\beta}{\sqrt{2}}\right|^2, \quad P(-) = \left|\frac{\alpha-\beta}{\sqrt{2}}\right|^2$$

Y Basis:

$$\langle +i|\psi\rangle = \frac{\alpha-i\beta}{\sqrt{2}}, \quad \langle -i|\psi\rangle = \frac{\alpha+i\beta}{\sqrt{2}}$$

$$P(+i) = \left|\frac{\alpha-i\beta}{\sqrt{2}}\right|^2, \quad P(-i) = \left|\frac{\alpha+i\beta}{\sqrt{2}}\right|^2$$

Physical Meaning:

- **Change of basis**: Unitary rotation (H, $S^\dagger$H)
- **Amplitude projection**: Inner product with basis states
- **Equivalence**: Both methods give identical probabilities
- **Implementation**: Real hardware uses gate rotation + Z measurement

5.8 Code: Measure Any Basis (Level 6)

File: `code/foundations/06_measure_any_basis.py`

```python
#!/usr/bin/env python3
"""
Level 6: Measure in Any Basis - Generalized

Generalizes measurement to X, Y, or Z basis.
From task-specific (Z only) to general tool!

Learning goal: Understand that measurement basis matters!
"""

import numpy as np
from qiskit.quantum_info import Statevector


def measure_basis(psi, basis='Z'):
    """
```

```python
    Measure state in any basis (X, Y, or Z).

    Args:
        psi: Statevector
        basis: 'Z', 'X', or 'Y'

    Returns:
        Dictionary of probabilities
    """
    if basis == 'Z':
        # Z basis: |0>, |1>
        prob_0 = np.abs(psi[0])**2
        prob_1 = np.abs(psi[1])**2
        return {'0': prob_0, '1': prob_1}

    elif basis == 'X':
        # X basis: |+>, |->
        # Transform to X basis
        plus_amp = (psi[0] + psi[1]) / np.sqrt(2)
        minus_amp = (psi[0] - psi[1]) / np.sqrt(2)
        return {'+': np.abs(plus_amp)**2, '-': np.abs(minus_amp)**2}

    elif basis == 'Y':
        # Y basis: |+i>, |-i>
        # Transform to Y basis
        plus_y_amp = (psi[0] - 1j*psi[1]) / np.sqrt(2)
        minus_y_amp = (psi[0] + 1j*psi[1]) / np.sqrt(2)
        return {'+i': np.abs(plus_y_amp)**2, '-i': np.abs(minus_y_amp)**2}

    else:
        raise ValueError("Basis must be 'Z', 'X', or 'Y'")


# Test with |+> state in different bases
plus = Statevector.from_label('+')
print("Measuring |+> in different bases:")
print(f"   Z basis: {measure_basis(plus, 'Z')}")  # 50/50
print(f"   X basis: {measure_basis(plus, 'X')}")  # 100% +
print(f"   Y basis: {measure_basis(plus, 'Y')}")  # 50/50

# Test with |0> state
zero = Statevector.from_label('0')
print("\nMeasuring |0> in different bases:")
print(f"   Z basis: {measure_basis(zero, 'Z')}")  # 100% 0
print(f"   X basis: {measure_basis(zero, 'X')}")  # 50/50
print(f"   Y basis: {measure_basis(zero, 'Y')}")  # 50/50
```

Listing 6: Measure in any basis (Level 6)

What This Code Does:

This code generalizes measurement from Z-basis only (Level 5) to any basis (X, Y, or Z). The `measure_basis()` function transforms the state to the chosen basis and computes probabilities. For X-basis, it projects onto $|\pm\rangle = \frac{|0\rangle \pm |1\rangle}{\sqrt{2}}$. For Y-basis, it projects onto $|\pm i\rangle = \frac{|0\rangle \pm i|1\rangle}{\sqrt{2}}$. Testing shows that $|+\rangle$ gives deterministic outcomes in X-basis (100% +) but random outcomes in Z and Y bases (50/50), demonstrating that measurement outcomes depend on the relationship between the state and measurement basis.

Physical Significance:

- **Basis choice**: Different measurements reveal different properties
- **Eigenstates**: $|+\rangle$ is an eigenstate of X (deterministic), not Z
- **Incompatible observables**: X and Z don't commute, can't both be certain
- **Basis transformation**: Change of basis is a unitary rotation
- **Progression**: Level 6 generalizes Level 5 (Z-only) to any basis

Expected Output:

```
Measuring |+> in different bases:
   Z basis: {'0': 0.5, '1': 0.5}
   X basis: {'+': 1.0, '-': 0.0}
   Y basis: {'+i': 0.5, '-i': 0.5}

Measuring |0> in different bases:
   Z basis: {'0': 1.0, '1': 0.0}
   X basis: {'+': 0.5, '-': 0.5}
   Y basis: {'+i': 0.5, '-i': 0.5}
```

Connection to Theory:
Measurement in basis $\{|b_1\rangle, |b_2\rangle\}$ gives:

- Probability: $P(b_i) = |\langle b_i|\psi\rangle|^2$
- X basis: $|\pm\rangle = \frac{|0\rangle \pm |1\rangle}{\sqrt{2}}$
- Y basis: $|\pm i\rangle = \frac{|0\rangle \pm i|1\rangle}{\sqrt{2}}$
- Z basis: $|0\rangle, |1\rangle$ (computational basis)

Physical Significance:

- **Born rule**: $P(i) = |\langle i|\psi\rangle|^2$ implemented directly
- **Z-basis**: Computational basis measurement (most common)
- **Probabilities**: Always sum to 1 (normalization)
- **Superposition**: $|+\rangle$ gives equal probabilities
- **Eigenstate**: $|0\rangle$ gives deterministic outcome

- **Measurement**: Irreversible collapse to $|0\rangle$ or $|1\rangle$

Expected Output:

```
Measuring |+> in Z basis:
   P(0) = 0.50 (50%)
   P(1) = 0.50 (50%)

Measuring |0> in Z basis:
   P(0) = 1.00 (100%)
   P(1) = 0.00 (0%)
```

5.9 Code: Apply X Gate (Level 7)

File: `code/foundations/07_apply_x_gate.py`

```python
#!/usr/bin/env python3
"""
Level 7: Apply X Gate - Task Specific

Applies X gate (NOT gate) to |0> state.
Task-specific: only X gate on one specific state.

Learning goal: See how gates transform states!
"""

from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

# Start with |0>
initial = Statevector.from_label('0')
print("Initial state: |0>")
print(initial)

# Create circuit and apply X gate
qc = QuantumCircuit(1)
qc.x(0)  # X gate (NOT)

# Apply circuit to state
final = initial.evolve(qc)
print("\nAfter X gate:")
print(final)

# X gate flipped |0> to |1>!
# That's why X is called the "NOT" gate
```

Listing 7: Apply X gate (Level 7)

What This Code Does:

This code demonstrates the X gate (quantum NOT gate) by initializing $|0\rangle$, creating a quantum circuit, applying the X gate, and evolving the state through the circuit. The result shows $|0\rangle$ flipped to $|1\rangle$, illustrating how unitary gates deterministically transform quantum states. This is the simplest example of gate operation on a single qubit.

Physical Significance:

- **X gate**: Quantum NOT gate (bit flip)
- **Unitary**: Reversible transformation
- **Bloch sphere**: 180° rotation around X-axis
- **Transformation**: $|0\rangle \to |1\rangle$
- **Deterministic**: Known input gives known output
- **Involutory**: $X^2 = I$ (applying twice returns to original)

Expected Output:

```
Initial state: |0>
Statevector([1.+0.j, 0.+0.j])

After X gate:
Statevector([0.+0.j, 1.+0.j])
```

5.10 Code: All Single Gates (Level 8)

File: `code/foundations/08_apply_all_single_gates.py`

```python
#!/usr/bin/env python3
"""
Level 8: Apply All Single-Qubit Gates

Applies all standard single-qubit gates (X, Y, Z, H, S, T)
to |0> state and shows the results.

Learning goal: Compare different gate transformations!
"""

from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

# Start with |0>
initial = Statevector.from_label('0')
print("Initial state: |0>")
print(initial)

# X Gate
qc_x = QuantumCircuit(1)
qc_x.x(0)
result_x = initial.evolve(qc_x)
print("\nAfter X gate:")
print(result_x)

# Y Gate
qc_y = QuantumCircuit(1)
qc_y.y(0)
result_y = initial.evolve(qc_y)
print("\nAfter Y gate:")
print(result_y)

# Z Gate
qc_z = QuantumCircuit(1)
qc_z.z(0)
result_z = initial.evolve(qc_z)
print("\nAfter Z gate:")
print(result_z)

# H Gate
qc_h = QuantumCircuit(1)
qc_h.h(0)
result_h = initial.evolve(qc_h)
print("\nAfter H gate:")
print(result_h)

# S Gate
qc_s = QuantumCircuit(1)
```

```python
qc_s.s(0)
result_s = initial.evolve(qc_s)
print("\nAfter S gate:")
print(result_s)

# T Gate
qc_t = QuantumCircuit(1)
qc_t.t(0)
result_t = initial.evolve(qc_t)
print("\nAfter T gate:")
print(result_t)
```

Listing 8: Apply all single-qubit gates (Level 8)

What This Code Does:

This code applies all standard single-qubit gates (X, Y, Z, H, S, T) to $|0\rangle$ and displays each result. By comparing the outputs, you can see how different gates transform the same initial state: X flips to $|1\rangle$, Y adds phase, Z does nothing to $|0\rangle$, H creates superposition, and S/T add phase rotations. This comprehensive demo lets you compare all single-qubit gate effects side-by-side.

Physical Significance:

- **X gate**: Bit flip ($|0\rangle \to |1\rangle$)
- **Y gate**: Bit + phase flip ($|0\rangle \to i|1\rangle$)
- **Z gate**: Phase flip ($|0\rangle \to |0\rangle$, $|1\rangle \to -|1\rangle$)
- **H gate**: Creates superposition ($|0\rangle \to |+\rangle$)
- **S gate**: $\pi/2$ phase rotation ($\sqrt{Z}$)
- **T gate**: $\pi/4$ phase rotation ($\sqrt{S}$)
- **Pauli gates**: X, Y, Z form basis for single-qubit operations
- **Clifford gates**: H, S are in Clifford group
- **Non-Clifford**: T gate enables universal quantum computation

Expected Output:

```
Initial state: |0>
Statevector([1.+0.j, 0.+0.j])

After X gate:
Statevector([0.+0.j, 1.+0.j])

After Y gate:
Statevector([0.+0.j, 0.+1.j])

After Z gate:
Statevector([1.+0.j, 0.+0.j])
```

```
After H gate:
Statevector([0.707+0.j, 0.707+0.j])

After S gate:
Statevector([1.+0.j, 0.+0.j])

After T gate:
Statevector([1.+0.j, 0.+0.j])
```

5.11 Code: Gate Transformer (Level 9)

File: `code/foundations/09_gate_transformer.py`

```python
#!/usr/bin/env python3
"""
Level 9: Gate Transformer - Generalized

Apply ANY sequence of gates to ANY initial state.
This is the most general single-qubit tool!

Learning goal: Complete generalization - universal transformer!
"""

from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector


def apply_gates(initial_state, gate_sequence):
    """
    Apply sequence of gates to initial state.

    Args:
        initial_state: Starting Statevector
        gate_sequence: List of gate names ('X', 'Y', 'Z', 'H', 'S', 'T')

    Returns:
        Final Statevector after all gates
    """
    # Create circuit
    qc = QuantumCircuit(1)

    # Apply each gate in sequence
    for gate in gate_sequence:
        if gate == 'X':
            qc.x(0)
        elif gate == 'Y':
            qc.y(0)
        elif gate == 'Z':
            qc.z(0)
        elif gate == 'H':
            qc.h(0)
        elif gate == 'S':
            qc.s(0)
        elif gate == 'T':
            qc.t(0)

    # Evolve state through circuit
    final_state = initial_state.evolve(qc)
    return final_state

```

```python

# Example 1: |0> -> X -> |1>
result1 = apply_gates(Statevector.from_label('0'), ['X'])
print("1. |0> --X--> |1>:")
print(result1)

# Example 2: |0> -> H -> |+>
result2 = apply_gates(Statevector.from_label('0'), ['H'])
print("\n2. |0> --H--> |+>:")
print(result2)

# Example 3: |0> -> H -> Z -> H -> |1>
result3 = apply_gates(Statevector.from_label('0'), ['H', 'Z', 'H'])
print("\n3. |0> --H Z H--> |1>:")
print(result3)

# Example 4: Custom sequence
result4 = apply_gates(Statevector.from_label('0'), ['H', 'S', 'T'])
print("\n4. |0> --H S T--> custom state:")
print(result4)
```

Listing 9: Apply any gate sequence (Level 9)

What This Code Does:

This code provides a universal gate transformer that applies any sequence of gates to any initial state. The `apply_gates()` function takes a state and a list of gate names (X, Y, Z, H, S, T), creates a circuit, applies each gate in order, and returns the final state. Examples show single gates (X, H), gate conjugation (HZH = X), and custom sequences (HST). This is the most general single-qubit tool, capable of implementing any unitary transformation.

Physical Significance:

- **Universality**: Any single-qubit transformation possible
- **Composability**: Gates can be chained in any order
- **Reusability**: One function handles all gate sequences
- **Group structure**: Single-qubit gates form SU(2) group
- **Progression**: Peak of Level 1-9 (most general tool)
- **Conjugation**: HZH = X (basis change trick)
- **Phase accumulation**: S and T add relative phases

Expected Output:

```
1. |0> --X--> |1>:
Statevector([0.+0.j, 1.+0.j])
```

```
2. |0> --H--> |+>:
Statevector([0.707+0.j, 0.707+0.j])

3. |0> --H Z H--> |1>:
Statevector([0.+0.j, 1.+0.j])

4. |0> --H S T--> custom state:
Statevector([0.707+0.j, 0.707+0.707j])
```

5.12 Exercises

1. **Eigenstates:** Find all eigenstates and eigenvalues of:
   (a) The X gate
   (b) The Y gate
   (c) The Z gate
   (d) The H gate

   Verify that eigenvalues have magnitude 1 (unitary property).

2. **Commutators:** Calculate the commutator $[A, B] = AB - BA$ for:
   (a) $[X, Y]$
   (b) $[Y, Z]$
   (c) $[Z, X]$
   (d) $[H, X]$

   What do non-zero commutators tell us about simultaneous measurement?

3. **Euler Angles:** Any single-qubit unitary can be written as:

$$U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$$

   Find $\alpha, \beta, \gamma, \delta$ for the H gate.

4. **Uncertainty Relation:** For state $|+\rangle$, calculate:
   (a) $\Delta X = \sqrt{\langle X^2 \rangle - \langle X \rangle^2}$
   (b) $\Delta Z = \sqrt{\langle Z^2 \rangle - \langle Z \rangle^2}$
   (c) Verify $\Delta X \Delta Z \geq \frac{1}{2} |\langle [X, Z] \rangle|$

5. **Phase Kickback:** Starting with $|0\rangle$, apply $H \to Z \to H$.
   (a) Track the phase at each step
   (b) Explain why the final state is $|1\rangle$ (not $|0\rangle$)
   (c) Relate this to quantum interference

6. **Density Matrix:** For $|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$:

   (a) Calculate $\rho = |\psi\rangle\langle\psi|$
   (b) Find the Bloch vector from ρ
   (c) Verify $\mathrm{Tr}(\rho) = 1$ and $\rho^2 = \rho$ (pure state)

7. **Code Extension:** Extend `gate_transformer.py` to:
   (a) Accept controlled gates (CNOT, CZ) for 2-qubit circuits
   (b) Visualize the circuit using Qiskit's `draw()`
   (c) Export the statevector to a file for later analysis

8. **No-Cloning Proof:** Assume a unitary U exists such that:

$$U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$$

   for any $|\psi\rangle$. Show this leads to a contradiction by considering:
   (a) $|\psi\rangle = |0\rangle$
   (b) $|\psi\rangle = |1\rangle$
   (c) $|\psi\rangle = |+\rangle$
   (d) Use linearity to derive the contradiction

9. **Measurement Backaction:** A qubit in state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ is measured:
   (a) If outcome is 0, what is the post-measurement state?
   (b) If we immediately measure again, what outcome is certain?
   (c) Why can't we recover α and β from one measurement?
   (d) How many copies do we need to estimate $|\alpha|^2$ to 1% accuracy?

10. **Physical Implementation:** Research one real qubit technology:
   (a) Superconducting qubits (transmon)
   (b) Trapped ions
   (c) Quantum dots
   (d) Photonics

   Explain how $|0\rangle$, $|1\rangle$, and superposition are physically realized.

5.13 Key Physical Concepts

- **Dirac notation**: Mathematical language encoding complete quantum state
- **Ket**: Complete description of physical system
- **Bra**: Measurement probe, dual vector
- **Qubit**: Two-level quantum system with superposition
- **Bloch sphere**: Geometric representation, intuitive visualization

- **Quantum gates**: Unitary rotations, reversible transformations
- **Measurement**: Probabilistic collapse, information extraction
- **Superposition**: Simultaneous existence in multiple states
- **Interference**: Wave-like addition of probability amplitudes
- **Phase**: Relative complex argument affecting interference
- **Normalization**: Total probability = 1
- **Orthogonality**: Mutually exclusive measurement outcomes

5.14 Why Single-Qubit Physics Matters

- **Foundation**: All multi-qubit phenomena build on single-qubit behavior
- **Control**: Single-qubit gates are most precise operations in quantum computers
- **Calibration**: Qubit characterization starts with single-qubit spectroscopy
- **Error correction**: Single-qubit errors are most common (decoherence)
- **Algorithm primitive**: All quantum algorithms use single-qubit rotations
- **Readout**: Measurement is single-qubit operation
- **State preparation**: Initialization is single-qubit operation

5.15 Real-World Applications

- **Quantum sensing**: Single qubits as ultra-sensitive magnetometers
- **Atomic clocks**: Qubit transitions define time standards
- **MRI**: Nuclear spin qubits in medical imaging
- **Quantum communication**: Single-photon qubits for secure key distribution
- **Simulation**: Single-qubit dynamics model molecular spin systems
- **Metrology**: Quantum-enhanced precision measurements
- **Fundamental tests**: Bell tests, Leggett-Garg inequalities